Act as a Principal SaaS Architect, Business Operations Systems Engineer, POS Architect, ERP Engineer, JavaScript Developer, Security Engineer, and Product Designer.

Your task is to build Layer 2 of an AI Business Decision Intelligence Platform as a standalone HTML application.

The layer is called:

BUSINESS OPERATIONS LAYER

This layer depends on the previously designed Data Acquisition Layer.

Do not build Business Intelligence, Digital Twin, simulation, or AI Decision Intelligence features yet.

Purpose

The application must allow businesses to perform and record their daily operations.

Its primary question is:

"What is happening inside the business right now?"

The layer must generate accurate, structured operational data for future intelligence layers.

Existing Architecture Compatibility

Use the existing platform conventions:

window.INFINICUS.bus.register(route, handler, schema)

window.INFINICUS.bus.call(route, payload)

window.INFINICUS.bus.emit(event, data)

window.INFINICUS.bus.on(event, listener)

All operational modules must communicate through the EngineBus.

Do not modify or recreate the existing simulation engine.

Dependency

The Business Operations Layer must send every accepted operational record into the Data Acquisition Layer using standardized business events.

Expected ingestion route:

data.event.ingest

The operational layer must not create a second independent data-validation system.

It may perform user-interface validation, but canonical validation, normalization, duplicate detection, and storage must remain in the Data Acquisition Layer.

Required Modules

Build:

1. Business workspace
2. Branch management
3. Product and service catalog
4. Sales and order management
5. Payment records
6. Cash register
7. Inventory operations
8. Expense management
9. Customer operations
10. Supplier management
11. Purchase orders
12. Employee profiles
13. Roles and permissions
14. Shift management
15. Task management
16. Daily closing
17. Audit logging
18. Export and backup

Operational Flow

The core sales flow must be:

Select business → Select branch → Create order → Add products or services → Select customer → Calculate subtotal, discount, tax, and total → Record payment → Complete order → Reduce inventory → Update customer purchase history → Update cash register → Emit standardized business events

Product Structure

Each product or service must contain:

{ productId, businessId, name, type, sku, category, sellingPrice, costPrice, currency, stockTracked, taxRate, status }

Order Structure

Each order must contain:

{ orderId, businessId, branchId, customerId, status, items, subtotal, discount, tax, total, currency, paymentStatus, createdAt }

Inventory Rule

Inventory must use movement records.

Never directly overwrite stock without creating an inventory movement.

Supported movement types:

inventory.received inventory.sold inventory.consumed inventory.wasted inventory.damaged inventory.transferred inventory.returned inventory.adjusted

Required EngineBus Routes

Register the following route families:

operations.workspace. *operations.branch.* operations.product. *operations.order.* operations.payment. *operations.cash.* operations.inventory. *operations.expense.* operations.customer. *operations.supplier.* operations.purchaseOrder. *operations.employee.* operations.shift. *operations.task.* operations.closing. *operations.audit.*

Use explicit route names and strict payload schemas.

Required Events

Emit:

operation.order.created operation.order.completed operation.order.cancelled operation.order.refunded

operation.payment.completed operation.payment.refunded

operation.inventory.received operation.inventory.consumed operation.inventory.adjusted operation.inventory.low

operation.expense.created operation.expense.approved

operation.customer.created operation.customer.purchase.completed

operation.purchase_order.created operation.purchase_order.received

operation.shift.started operation.shift.ended

operation.task.created operation.task.completed

operation.day.closed

Each event must also be passed to data.event.ingest.

Data Integrity

Implement:

- Unique IDs
- Business ownership checks
- Branch ownership checks
- Duplicate order-completion prevention
- Stock validation
- Numeric bounds
- Payment-total reconciliation
- Cash-register reconciliation
- Immutable completed orders
- Refund and reversal records
- Audit trails
- Safe HTML rendering
- Input sanitization
- Error envelopes
- Transaction-safe operations where possible

User Interface

Use:

- Black background
- Neon green accent
- White typography
- IBM Plex Sans
- IBM Plex Mono
- Mobile-first responsive layout
- Desktop left navigation
- Accessible forms
- Fast order-entry workflow
- Clear confirmation messages
- Visible validation errors
- Searchable tables
- Offline-capable local storage foundation

Primary mobile navigation:

Home Sell Orders Inventory More

Expanded sections inside More:

Expenses Customers Suppliers Team Tasks Closing Settings

Storage

Use the storage adapter provided by the Data Acquisition Layer.

For standalone development, provide a compatible local adapter using:

- localStorage for settings
- IndexedDB for orders, inventory movements, and audit logs

Keep a clean interface for future Firestore or Supabase integration.

Build Method

Build in small executable blocks.

For each block:

1. State its purpose.
2. State dependencies.
3. Provide only the code required for that block.
4. Explain exactly where to insert it.
5. Provide a manual test procedure.
6. Review security and data-integrity risks.
7. Stop after completing the block.

Do not regenerate the entire project after every block.

Do not remove or rewrite previously working blocks.

First Response

Do not begin coding immediately.

First provide:

1. Updated Layer 2 architecture.
2. Module dependency map.
3. Canonical operational schemas.
4. EngineBus route map.
5. Event map.

6. Storage strategy.
7. Permission model.
8. Audit strategy.
9. Exact development sequence.

After completing the architecture review, begin with Block 1 only.